

MANUAL TÉCNICO - BCR DATA PIPELINE

Sistema de Ingesta de Datos con Arquitectura Medallón

ÍNDICE

- 1. [Introducción](#)
- 2. [Arquitectura del Sistema](#)
- 3. [Descripción Detallada por Módulo](#)
- 4. [Guía de Instalación](#)
- 5. [Guía de Uso](#)
- 6. [Sistema de Auditoría](#)

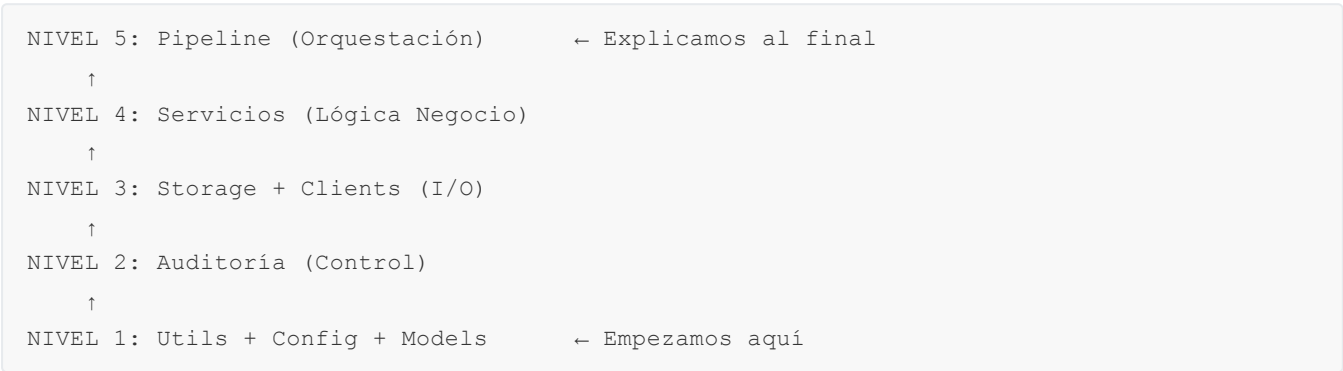
1. INTRODUCCIÓN

1.1 Propósito

Pipeline de ingesta de datos del Banco Central de Reserva del Perú (BCRP) con arquitectura medallón y gobernanza completa.

1.2 Enfoque Pedagógico: Bottom-Up

Este manual explica el código desde los cimientos hacia arriba:



Ventajas:

- Comprensión gradual sin sorpresas
- Entender dependencias antes de complejidad
- Facilita debugging y onboarding

1.3 Características Técnicas

- **Arquitectura:** Medallón (Bronze implementada)
- **Fuente:** API REST BCRP
- **Lenguaje:** Python 3.9+
- **Patrones:** Repository, Service Layer, Template Method
- **Gobernanza:** Auditoría completa + Quality Gates

2. ARQUITECTURA DEL SISTEMA

2.1 Estructura del Proyecto

[illegible]

```
├─ requirements.txt
└─ main.py
```

2.2 Mapa de Dependencias

```
Externas: requests, pydantic, pyyaml, tenacity, python-dotenv
    ↓
NIVEL 1: utils/logger, utils/dates, config/settings, models/*
    ↓
NIVEL 2: audit/audit_logger, audit/control_manager
    ↓
NIVEL 3: clients/*, storage/data_lake
    ↓
NIVEL 4: services/ingestion_service
    ↓
NIVEL 5: pipelines/bronze_pipeline
```

3. DESCRIPCIÓN DETALLADA POR MÓDULO

NIVEL 1: CIMIENTOS (Sin dependencias internas)

3.1 `app/utils/logger.py` - Logging Estructurado

Propósito: Producir logs JSON parseables para debugging y monitoreo.

Código:

```
import logging
import json
from datetime import datetime

class StructuredLogger:
    def __init__(self, name: str, level: str = "INFO"):
        self.logger = logging.getLogger(name)
        self.logger.setLevel(getattr(logging, level))

        handler = logging.StreamHandler()
        handler.setFormatter(logging.Formatter('%(message)s'))
        self.logger.addHandler(handler)

    def _format_message(self, level: str, message: str, **kwargs) -> str:
        log_entry = {
            "timestamp": datetime.now().isoformat(),
            "level": level,
            "message": message,
            **kwargs
        }
        return json.dumps(log_entry)

    def info(self, message: str, **kwargs):
```

```

self.logger.info(self._format_message("INFO", message, **kwargs))

def error(self, message: str, **kwargs):
    self.logger.error(self._format_message("ERROR", message, **kwargs))

```

Funcionamiento:

1. Crea logger con nombre identificador
2. Formatea mensajes como JSON con timestamp
3. Agrega contexto vía kwargs

Uso:

```

logger = StructuredLogger("BCRClient")
logger.info("Data fetched", series="PD04637PD", records=85)
# Output: {"timestamp":"2026-05-01T10:30:00","level":"INFO",
#          "message":"Data fetched","series":"PD04637PD","records":85}

```

Ventaja: Compatible con herramientas de análisis (ELK, Splunk).

3.2 `app/utls/dates.py` - Utilidades de Fechas

Propósito: Generar fechas en formato BCR API (YYYY-M-D sin ceros).

Código:

```

from datetime import datetime, timedelta
from typing import Tuple

def get_date_range(days_back: int = 30) -> Tuple[str, str]:
    """Generate date range in BCR format (YYYY-M-D)"""
    end_date = datetime.now()
    start_date = end_date - timedelta(days=days_back)

    return (
        start_date.strftime("%Y-%m-%d"),
        end_date.strftime("%Y-%m-%d")
    )

def get_partition_path(date: datetime) -> str:
    """Generate partition path YYYY/MM/DD"""
    return date.strftime("%Y/%m/%d")

```

Funcionamiento:

- `get_date_range()`: Formato BCR sin ceros (`%m`, `%d`)
- `get_partition_path()`: Formato directorios con ceros

Ejemplo:

```
get_date_range(30)
# Retorna: ("2026-4-1", "2026-5-1")

get_partition_path(datetime(2026, 5, 1))
# Retorna: "2026/05/01"
```

3.3 `app/config/settings.py` - Configuración Centralizada

Propósito: Cargar config desde `.env` y `config.yaml` con precedencia.

Código:

```
import os
import yaml
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()

class Settings:
    def __init__(self):
        self.config = self._load_config()

    def _load_config(self):
        with open(Path("config.yaml"), 'r') as f:
            return yaml.safe_load(f)

    @property
    def bcr_base_url(self) -> str:
        return os.getenv("BCR_BASE_URL", self.config['api']['base_url'])

    @property
    def timeout(self) -> int:
        return int(os.getenv("BCR_TIMEOUT", self.config['api']['timeout']))

    @property
    def data_lake_path(self) -> Path:
        return Path(os.getenv("DATA_LAKE_PATH", self.config['data_lake']['base_path']))

settings = Settings() # Singleton
```

Precedencia:

1. Variables `.env` (overrides)
2. `config.yaml` (defaults)

Uso:

```
from app.config.settings import settings
url = settings.bcr_base_url
```

3.4 app/models/schemas.py - Modelos de Datos

Propósito: Validación automática con Pydantic.

Código:

```
from typing import Any, Dict, Optional, List
from datetime import datetime
from pydantic import BaseModel, Field

class APIMetadata(BaseModel):
    source: str
    endpoint: str
    ingestion_date: datetime = Field(default_factory=datetime.now)
    status: str
    error_message: Optional[str] = None
    execution_time_ms: Optional[float] = None

class BronzeRecord(BaseModel):
    metadata: APIMetadata
    data: Dict[str, Any]

class BCRResponse(BaseModel):
    config: Dict[str, Any]
    periods: List[Dict[str, Any]]
```

Uso:

```
# Validación automática
metadata = APIMetadata(
    source="bcr",
    endpoint="exchange_rate/PD04637PD",
    status="success"
)

# Serialización JSON
record = BronzeRecord(metadata=metadata, data={...})
json_data = record.model_dump(mode='json')
```

Ventaja: Type hints + validación runtime + serialización.

3.5 app/models/audit_schemas.py - Modelos Auditoría

Código:

```
from typing import Optional, Dict, Any, List
from datetime import datetime
from pydantic import BaseModel, Field
from enum import Enum
```

```

class ExecutionStatus(str, Enum):
    SUCCESS = "success"
    FAILED = "failed"
    PARTIAL = "partial"
    RUNNING = "running"

class DataQualityStatus(str, Enum):
    PASSED = "passed"
    FAILED = "failed"
    WARNING = "warning"

class AuditLog(BaseModel):
    audit_id: str = Field(default_factory=lambda:
datetime.now().strftime("%Y%m%d_%H%M%S_%f"))
    pipeline_name: str
    execution_id: str
    timestamp: datetime = Field(default_factory=datetime.now)
    status: ExecutionStatus
    duration_seconds: Optional[float] = None
    records_processed: int = 0
    records_success: int = 0
    records_failed: int = 0
    error_details: Optional[List[Dict[str, Any]]] = None
    metadata: Dict[str, Any] = Field(default_factory=dict)

class DataQualityCheck(BaseModel):
    check_id: str
    check_name: str
    check_type: str
    status: DataQualityStatus
    dataset: str
    records_checked: int
    records_passed: int
    records_failed: int
    failure_rate: float

class ExecutionControl(BaseModel):
    execution_id: str = Field(default_factory=lambda:
datetime.now().strftime("%Y%m%d_%H%M%S"))
    pipeline_name: str
    start_time: datetime = Field(default_factory=datetime.now)
    end_time: Optional[datetime] = None
    status: ExecutionStatus = ExecutionStatus.RUNNING
    input_parameters: Dict[str, Any]
    output_summary: Optional[Dict[str, Any]] = None
    quality_checks: List[DataQualityCheck] = Field(default_factory=list)
    audit_logs: List[AuditLog] = Field(default_factory=list)

```

Lógica Quality Check:

```

if failure_rate == 0:
    status = PASSED
elif failure_rate < 0.05: # 5% threshold
    status = WARNING
else:
    status = FAILED

```

NIVEL 2: AUDITORÍA (Usa: models + utils)

3.6 app/audit/audit_logger.py - Persistencia Auditoría

Propósito: Escribir logs de auditoría en disco con particionamiento.

Código Principal:

```

import json
from pathlib import Path
from datetime import datetime
from app.models.audit_schemas import AuditLog, DataQualityCheck
from app.utils.logger import StructuredLogger

class AuditLogger:
    def __init__(self, audit_path: Path):
        self.audit_path = audit_path
        self.logger = StructuredLogger(self.__class__.__name__)
        self._ensure_audit_structure()

    def _ensure_audit_structure(self):
        """Crea: executions/, quality_checks/, errors/, metrics/"""
        directories = [
            self.audit_path / "executions",
            self.audit_path / "quality_checks",
            self.audit_path / "errors",
            self.audit_path / "metrics"
        ]
        for directory in directories:
            directory.mkdir(parents=True, exist_ok=True)

    def log_execution(self, audit_log: AuditLog) -> Path:
        """Escribe: audit/executions/YYYY/MM/DD/{audit_id}.json"""
        date_partition = datetime.now().strftime("%Y/%m/%d")
        output_dir = self.audit_path / "executions" / date_partition
        output_dir.mkdir(parents=True, exist_ok=True)

        file_path = output_dir / f"{audit_log.audit_id}.json"

        with open(file_path, 'w', encoding='utf-8') as f:
            json.dump(audit_log.model_dump(mode='json'), f, indent=2, ensure_ascii=False)

        self.logger.info("Audit log created", audit_id=audit_log.audit_id)
        return file_path

```

```

def log_quality_check(self, quality_check: DataQualityCheck) -> Path:
    """Escribe: audit/quality_checks/YYYY/MM/DD/{check_id}.json"""
    date_partition = datetime.now().strftime("%Y/%m/%d")
    output_dir = self.audit_path / "quality_checks" / date_partition
    output_dir.mkdir(parents=True, exist_ok=True)

    file_path = output_dir / f"{quality_check.check_id}.json"

    with open(file_path, 'w', encoding='utf-8') as f:
        json.dump(quality_check.model_dump(mode='json'), f, indent=2)

    return file_path

def get_execution_summary(self, date: datetime = None) -> Dict[str, Any]:
    """Genera resumen diario de ejecuciones"""
    if date is None:
        date = datetime.now()

    date_partition = date.strftime("%Y/%m/%d")
    execution_dir = self.audit_path / "executions" / date_partition

    if not execution_dir.exists():
        return {"date": date.isoformat(), "total_executions": 0}

    logs = []
    for file in execution_dir.glob("*.json"):
        with open(file, 'r') as f:
            logs.append(json.load(f))

    return {
        "date": date.isoformat(),
        "total_executions": len(logs),
        "successful": sum(1 for log in logs if log['status'] == 'success'),
        "failed": sum(1 for log in logs if log['status'] == 'failed'),
        "total_records_processed": sum(log.get('records_processed', 0) for log in
logs)
    }

```

Estructura Generada:

```

audit/
├─ executions/2026/05/01/20260501_103000_123456.json
├─ quality_checks/2026/05/01/completeness_PD04637PD_20260501.json
└─ errors/2026/05/01/error_20260501_103010.json

```

3.7 app/audit/control_manager.py - Gestor de Control

Propósito: Orquestar control de ejecuciones y quality checks.

Código Principal:

```
from typing import Dict, Any, List, Optional
from datetime import datetime
from app.models.audit_schemas import (
    ExecutionControl, AuditLog, DataQualityCheck,
    ExecutionStatus, DataQualityStatus
)
from app.audit.audit_logger import AuditLogger
from app.utils.logger import StructuredLogger

class ControlManager:
    def __init__(self, audit_logger: AuditLogger):
        self.audit_logger = audit_logger
        self.logger = StructuredLogger(self.__class__.__name__)
        self.current_execution: Optional[ExecutionControl] = None

    def start_execution(self, pipeline_name: str, input_parameters: Dict[str, Any]) ->
ExecutionControl:
        """Inicia control de ejecución"""
        self.current_execution = ExecutionControl(
            pipeline_name=pipeline_name,
            input_parameters=input_parameters
        )

        self.logger.info("Execution started",
execution_id=self.current_execution.execution_id)
        return self.current_execution

    def end_execution(self, status: ExecutionStatus, output_summary: Dict[str, Any]) ->
ExecutionControl:
        """Finaliza y registra ejecución"""
        if not self.current_execution:
            raise ValueError("No active execution")

        self.current_execution.end_time = datetime.now()
        self.current_execution.status = status
        self.current_execution.output_summary = output_summary

        duration = (self.current_execution.end_time -
self.current_execution.start_time).total_seconds()

        audit_log = AuditLog(
            pipeline_name=self.current_execution.pipeline_name,
            execution_id=self.current_execution.execution_id,
            status=status,
            duration_seconds=duration,
            records_processed=output_summary.get('total_series', 0),
            records_success=output_summary.get('successful', 0),
```

```

        records_failed=output_summary.get('failed', 0),
        metadata=self.current_execution.input_parameters
    )

    self.audit_logger.log_execution(audit_log)
    self.current_execution.audit_logs.append(audit_log)

    return self.current_execution

def perform_quality_check(
    self,
    check_name: str,
    check_type: str,
    dataset: str,
    data: List[Dict[str, Any]],
    validation_func: callable
) -> DataQualityCheck:
    """Ejecuta verificación de calidad"""

    records_checked = len(data)
    validation_results = [validation_func(record) for record in data]
    records_passed = sum(validation_results)
    records_failed = records_checked - records_passed
    failure_rate = records_failed / records_checked if records_checked > 0 else 0

    # Determinar status
    if failure_rate == 0:
        status = DataQualityStatus.PASSED
    elif failure_rate < 0.05:
        status = DataQualityStatus.WARNING
    else:
        status = DataQualityStatus.FAILED

    quality_check = DataQualityCheck(
        check_id=f"{check_name}_{datetime.now().strftime('%Y%m%d_%H%M%S')}",
        check_name=check_name,
        check_type=check_type,
        status=status,
        dataset=dataset,
        records_checked=records_checked,
        records_passed=records_passed,
        records_failed=records_failed,
        failure_rate=failure_rate
    )

    self.audit_logger.log_quality_check(quality_check)

    if self.current_execution:
        self.current_execution.quality_checks.append(quality_check)

    return quality_check

```

Flujo de Uso:

1. `start_execution()` → Crea `ExecutionControl` (status=RUNNING)
2. `perform_quality_check()` → Valida datos, registra
3. `end_execution()` → Calcula duración, crea `AuditLog`, persiste

NIVEL 3: I/O (Usa: config + utils + models)

3.8 `app/clients/base_client.py` - Cliente Base

Propósito: Clase abstracta para clientes API con retry logic.

Código:

```
from abc import ABC, abstractmethod
from typing import Dict, Any, Optional
import requests
from tenacity import retry, stop_after_attempt, wait_exponential
from app.utils.logger import StructuredLogger

class BaseAPIClient(ABC):
    def __init__(self, base_url: str, timeout: int = 30, max_retries: int = 3):
        self.base_url = base_url
        self.timeout = timeout
        self.max_retries = max_retries
        self.logger = StructuredLogger(self.__class__.__name__)

    @retry(
        stop=stop_after_attempt(3),
        wait=wait_exponential(multiplier=2, min=2, max=10)
    )
    def _make_request(
        self,
        endpoint: str,
        method: str = "GET",
        params: Optional[Dict] = None
    ) -> Dict[str, Any]:
        """Ejecuta HTTP request con retry automático"""
        url = f"{self.base_url}/{endpoint}"

        try:
            self.logger.info("API request initiated", url=url, method=method)

            response = requests.request(
                method=method,
                url=url,
                params=params,
                timeout=self.timeout
            )
            response.raise_for_status()

            self.logger.info("API request successful", status_code=response.status_code)
            return response.json()
```

```

        except requests.exceptions.RequestException as e:
            self.logger.error("API request failed", error=str(e), url=url)
            raise

    @abstractmethod
    def fetch_data(self, **kwargs) -> Dict[str, Any]:
        """Implementar en subclasses"""
        pass

```

Retry Logic:

```

Intento 1: Inmediato
Intento 2: Espera 2s
Intento 3: Espera 4s
Fallo: Lanza excepción

```

Patrón: Template Method

3.9 app/clients/bcr_client.py - Cliente BCR

Propósito: Cliente específico para API BCRP.

Código:

```

from typing import Dict, Any
from app.clients.base_client import BaseAPIClient
from app.utils.dates import get_date_range

class BCRClient(BaseAPIClient):
    def fetch_data(
        self,
        series_code: str,
        start_date: str = None,
        end_date: str = None,
        format: str = "json"
    ) -> Dict[str, Any]:
        """
        Fetch data from BCR API

        Endpoint format: {code}/{format}/{start_date}/{end_date}
        Example: PD04637PD/json/2026-1-1/2026-4-30
        """
        if not start_date or not end_date:
            start_date, end_date = get_date_range(days_back=30)

        endpoint = f"{series_code}/{format}/{start_date}/{end_date}"

        return self._make_request(endpoint)

    def fetch_multiple_series(

```

```

        self,
        series_codes: list,
        start_date: str = None,
        end_date: str = None,
        format: str = "json"
    ) -> Dict[str, Any]:
        """Fetch múltiples series en una request"""
        codes = "-".join(series_codes)

        if not start_date or not end_date:
            start_date, end_date = get_date_range(days_back=30)

        endpoint = f"{codes}/{format}/{start_date}/{end_date}"

        return self._make_request(endpoint)

```

Nota: Formato BCR requiere `YYYY-M-D` sin ceros.

3.10 `app/storage/data_lake.py` - Data Lake

Propósito: Escritura de datos Bronze con particionamiento.

Código:

```

import json
from pathlib import Path
from datetime import datetime
from app.models.schemas import BronzeRecord
from app.utils.logger import StructuredLogger

class DataLake:
    def __init__(self, base_path: Path):
        self.base_path = base_path
        self.logger = StructuredLogger(self.__class__.__name__)

    def write_bronze(
        self,
        record: BronzeRecord,
        source: str,
        endpoint: str,
        partition_date: datetime = None
    ) -> Path:
        """
        Escribe record en Bronze con particionamiento

        Estructura: bronze/{source}/{endpoint}/YYYY/MM/DD/{timestamp}.json
        Ejemplo: bronze/bcr/exchange_rate/2026/05/01/20260501_103045_123456.json
        """
        if partition_date is None:
            partition_date = datetime.now()

        partition_path = partition_date.strftime("%Y/%m/%d")

```

```

output_dir = (
    self.base_path /
    "bronze" /
    source /
    endpoint /
    partition_path
)
output_dir.mkdir(parents=True, exist_ok=True)

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
file_path = output_dir / f"{timestamp}.json"

with open(file_path, 'w', encoding='utf-8') as f:
    json.dump(record.model_dump(mode='json'), f, indent=2, ensure_ascii=False)

self.logger.info("Bronze record written", file_path=str(file_path))

return file_path

```

Ventajas Particionamiento:

- Eliminación eficiente de datos antiguos
- Queries optimizadas por fecha
- Compatible con herramientas big data (Spark, Hive)

NIVEL 4: SERVICIOS (Orquesta: clients + storage + audit)

3.11 `app/services/ingestion_service.py` - Servicio de Ingesta

Propósito: Coordinar ingesta desde API hasta Data Lake con validación.

Código Principal:

```

from typing import Dict, Any, Optional
from datetime import datetime
import time
from app.clients.bcr_client import BCRClient
from app.storage.data_lake import DataLake
from app.models.schemas import BronzeRecord, APIMetadata
from app.audit.control_manager import ControlManager
from app.utils.logger import StructuredLogger

class IngestionService:
    def __init__(
        self,
        client: BCRClient,
        data_lake: DataLake,
        control_manager: Optional[ControlManager] = None
    ):
        self.client = client

```

```

self.data_lake = data_lake
self.control_manager = control_manager
self.logger = StructuredLogger(self.__class__.__name__)

def ingest_series(
    self,
    series_code: str,
    start_date: str = None,
    end_date: str = None
) -> Dict[str, Any]:
    """Ingesta completa con auditoría"""

    start_time = time.time()

    try:
        # 1. Fetch data
        data = self.client.fetch_data(
            series_code=series_code,
            start_date=start_date,
            end_date=end_date
        )

        # 2. Quality check
        if self.control_manager:
            self._validate_data_quality(series_code, data)

        execution_time = (time.time() - start_time) * 1000

        # 3. Build metadata
        metadata = APIMetadata(
            source="bcr",
            endpoint=f"exchange_rate/{series_code}",
            ingestion_date=datetime.now(),
            status="success",
            execution_time_ms=execution_time
        )

        # 4. Create Bronze record
        record = BronzeRecord(metadata=metadata, data=data)

        # 5. Write to Data Lake
        file_path = self.data_lake.write_bronze(
            record=record,
            source="bcr",
            endpoint="exchange_rate"
        )

        self.logger.info("Series ingestion completed", series_code=series_code)

    return {
        "status": "success",
        "series_code": series_code,
        "file_path": str(file_path),
    }

```

```

        "execution_time_ms": execution_time,
        "records_count": len(data.get('periods', []))
    }

    except Exception as e:
        self.logger.error("Series ingestion failed", series_code=series_code,
error=str(e))

        if self.control_manager:
            self.control_manager.log_pipeline_error(
                error_type="IngestionError",
                error_message=str(e),
                context={"series_code": series_code}
            )

        return {
            "status": "error",
            "series_code": series_code,
            "error": str(e)
        }

def _validate_data_quality(self, series_code: str, data: Dict[str, Any]):
    """Valida completitud de datos"""

    def validate_record(record: Dict) -> bool:
        return (
            'name' in record and
            'values' in record and
            len(record['values']) > 0 and
            record['values'][0] is not None
        )

    periods = data.get('periods', [])

    if periods:
        self.control_manager.perform_quality_check(
            check_name=f"completeness_{series_code}",
            check_type="completeness",
            dataset=series_code,
            data=periods,
            validation_func=validate_record
        )

```

Flujo:

```

ingest_series()
|
├─ 1. client.fetch_data()           → Obtiene JSON
├─ 2. _validate_data_quality()      → Quality check
├─ 3. APIMetadata construction     → Metadata
├─ 4. BronzeRecord creation        → Empaqueta
└─ 5. data_lake.write_bronze()     → Persiste

```

NIVEL 5: ORQUESTACIÓN (Usa: todos los anteriores)

3.12 app/pipelines/bronze/bronze_pipeline.py - Pipeline Bronze

Propósito: Orquestador principal del pipeline completo.

Código:

```
from typing import List, Dict, Any
from app.clients.bcr_client import BCRCClient
from app.storage.data_lake import DataLake
from app.services.ingestion_service import IngestionService
from app.audit.audit_logger import AuditLogger
from app.audit.control_manager import ControlManager
from app.config.settings import settings
from app.models.audit_schemas import ExecutionStatus
from app.utils.logger import StructuredLogger

class BronzePipeline:
    def __init__(self):
        self.logger = StructuredLogger(self.__class__.__name__)
        self.client = BCRCClient(base_url=settings.bcr_base_url)
        self.data_lake = DataLake(base_path=settings.data_lake_path)

        # Auditoría
        audit_path = settings.data_lake_path / "audit"
        self.audit_logger = AuditLogger(audit_path=audit_path)
        self.control_manager = ControlManager(audit_logger=self.audit_logger)

        # Servicio
        self.ingestion_service = IngestionService(
            self.client,
            self.data_lake,
            self.control_manager
        )

    def run(
        self,
        series_codes: List[str],
        start_date: str = None,
        end_date: str = None
    ) -> Dict[str, Any]:
        """Ejecuta pipeline completo con auditoría"""

        # 1. Iniciar control
        execution = self.control_manager.start_execution(
            pipeline_name="bronze_exchange_rate",
            input_parameters={
                "series_codes": series_codes,
                "start_date": start_date,
```

```

        "end_date": end_date
    }
)

self.logger.info("Bronze pipeline started", execution_id=execution.execution_id)

# 2. Procesar series
results = []
errors = []

for code in series_codes:
    result = self.ingestion_service.ingest_series(
        series_code=code,
        start_date=start_date,
        end_date=end_date
    )
    results.append(result)

    if result['status'] == 'error':
        errors.append({
            "series_code": code,
            "error": result.get('error')
        })

# 3. Determinar status final
success_count = sum(1 for r in results if r['status'] == 'success')

if success_count == len(results):
    final_status = ExecutionStatus.SUCCESS
elif success_count == 0:
    final_status = ExecutionStatus.FAILED
else:
    final_status = ExecutionStatus.PARTIAL

# 4. Output summary
output_summary = {
    "total_series": len(results),
    "successful": success_count,
    "failed": len(results) - success_count,
    "errors": errors,
    "details": results
}

# 5. Finalizar control
self.control_manager.end_execution(
    status=final_status,
    output_summary=output_summary
)

# 6. Generar reporte
execution_report = self.control_manager.get_execution_report()

self.logger.info("Bronze pipeline completed", execution_id=execution.execution_id)

```

```
return {  
    "pipeline": "bronze",  
    "execution_id": execution.execution_id,  
    **output_summary,  
    "execution_report": execution_report  
}
```

Flujo Completo:

```
run()  
|  
|→ 1. start_execution()           → Inicia control  
|  
|→ 2. Para cada serie:  
|    ↳ ingest_series()  
|        ↳ fetch_data()  
|        ↳ quality_check()  
|        ↳ write_bronze()  
|  
|→ 3. Determinar status  
|    ↳ 100% success → SUCCESS  
|    ↳ 100% failed  → FAILED  
|    ↳ Mixto        → PARTIAL  
|  
|→ 4. end_execution()           → Finaliza, registra  
|  
|→ 5. Retorna resumen completo
```

4. GUÍA DE INSTALACIÓN

4.1 Prerrequisitos

- Python 3.9+
- pip

4.2 Instalación

```
# Crear entorno virtual  
python -m venv venv  
source venv/bin/activate # Linux/Mac  
# venv\Scripts\activate  # Windows  
  
# Instalar dependencias  
pip install -r requirements.txt
```

4.3 Configuración

Crear `.env`:

```
cp .env.example .env
```

Contenido `.env`:

```
BCR_BASE_URL=https://estadisticas.bcrp.gob.pe/estadisticas/series/api
BCR_TIMEOUT=30
DATA_LAKE_PATH=./data
```

`config.yaml`:

```
api:
  base_url: "https://estadisticas.bcrp.gob.pe/estadisticas/series/api"
  timeout: 30
  max_retries: 3

data_lake:
  base_path: "./data"
  layers:
    bronze: "bronze"
```

4.4 Verificación

```
python -c "from app.config.settings import settings; print(settings.bcr_base_url)"
```

5. GUÍA DE USO

5.1 Ejecución Básica

```
python main.py
```

5.2 `main.py`

```
from app.pipelines.bronze.bronze_pipeline import BronzePipeline
import json

def main():
    exchange_rate_codes = [
        "PD04637PD", # TC Interbancario Compra
        "PD04638PD", # TC Interbancario Venta
        "PD04639PD", # TC Sistema bancario Compra
        "PD04640PD", # TC Sistema bancario Venta
    ]
```

```

pipeline = BronzePipeline()

result = pipeline.run(
    series_codes=exchange_rate_codes,
    start_date="2026-1-1",
    end_date="2026-4-30"
)

print(f"Execution ID: {result['execution_id']}")
print(f"Total Series: {result['total_series']}")
print(f"Successful: {result['successful']}")
print(json.dumps(result['execution_report'], indent=2, default=str))

if __name__ == "__main__":
    main()

```

5.3 Uso Programático

```

from app.pipelines.bronze.bronze_pipeline import BronzePipeline

pipeline = BronzePipeline()

result = pipeline.run(
    series_codes=["PD04637PD"],
    start_date="2026-4-1",
    end_date="2026-4-30"
)

print(f"Status: {result['execution_report']['status']}")

```

5.4 Análisis de Resultados

Ver datos ingestados:

```

find data/bronze -name "*.json" -type f
cat $(find data/bronze -name "*.json" -type f | tail -1) | python -m json.tool

```

Ver auditoría:

```

find data/audit/executions -name "*.json"
cat $(find data/audit/executions -name "*.json" | tail -1) | python -m json.tool

```

Ver quality checks:

```

find data/audit/quality_checks -name "*.json"

```

6. SISTEMA DE AUDITORÍA

6.1 Estructura

```
data/audit/  
├── executions/YYYY/MM/DD/      # Logs de ejecución  
├── quality_checks/YYYY/MM/DD/  # Validaciones calidad  
├── errors/YYYY/MM/DD/         # Errores detallados  
└── metrics/                   # Métricas (futuro)
```

6.2 Tipos de Auditoría

Ejecución:

```
{  
  "audit_id": "20260501_103000_123456",  
  "pipeline_name": "bronze_exchange_rate",  
  "execution_id": "20260501_103000",  
  "status": "success",  
  "duration_seconds": 15.5,  
  "records_processed": 4,  
  "records_success": 4,  
  "records_failed": 0  
}
```

Quality Check:

```
{  
  "check_id": "completeness_PD04637PD_20260501",  
  "check_type": "completeness",  
  "status": "passed",  
  "records_checked": 85,  
  "records_passed": 85,  
  "failure_rate": 0.0  
}
```

6.3 Consultas

Resumen diario:

```
from app.audit.audit_logger import AuditLogger  
from pathlib import Path  
from datetime import datetime  
  
audit_logger = AuditLogger(audit_path=Path("data/audit"))  
summary = audit_logger.get_execution_summary(date=datetime(2026, 5, 1))  
  
print(f"Ejecuciones: {summary['total_executions']}")  
print(f"Exitosas: {summary['successful']}")
```

6.4 KPIs

Tasa de Éxito:

```
Success Rate = (Successful / Total) * 100
Objetivo: > 95%
```

Calidad de Datos:

```
Quality Score = (Checks Passed / Total Checks) * 100
Objetivo: > 98%
```

Tiempo de Ejecución:

```
Avg Time = Σ(duration_seconds) / Total
Objetivo: < 30s por serie
```

ANEXOS

A. Códigos de Series BCR

Código	Descripción
PD04637PD	TC Interbancario Compra
PD04638PD	TC Interbancario Venta
PD04639PD	TC Sistema bancario Compra
PD04640PD	TC Sistema bancario Venta
PD04643PD	TC 11:00 AM Compra
PD04644PD	TC 11:00 AM Venta

B. Glosario

Término	Definición
Bronze Layer	Datos RAW sin transformación
Silver Layer	Datos limpios y normalizados
Gold Layer	Datos agregados para consumo
Quality Gate	Punto de validación de calidad
Audit Trail	Registro de auditoría de operaciones
Retry Logic	Lógica de reintento ante fallos

Término	Definición
Partitioning	División de datos por fecha

C. Dependencias

```
requests==2.31.0
python-dotenv==1.0.0
pyyaml==6.0.1
pydantic==2.5.0
tenacity==8.2.3
```